Paging | Non-Contiguous Memory Allocation

1.Problem with Dynamic Partitioning (External Fragmentation)

Think of **RAM like a long shelf** in a storeroom.

- Different sized boxes (processes) are placed on the shelf.
- Over time, some boxes are removed.
- This leaves **gaps (holes)** of different sizes between boxes.

Now suppose:

- You have **two empty gaps of 1KB each**.
- Total free space = **2KB**.
- But the gaps are **not together**.

If a new box (process) needs **2KB in one piece**, you **cannot place it**, even though total space exists.

This is **External Fragmentation**
Compaction can fix it by rearranging boxes, but **moving boxes again and again is costly**.

So we need a **better, smarter mechanism**.

2.Idea Behind Paging

Now change the thinking.

Instead of keeping each box in **one single piece**, we **cut the box into smaller fixed-size pieces**.

Real-World Example: Bookshelf 📚

- Shelf = **RAM**
- Books = **Process**
- Pages = **Paging blocks**

Suppose:

- A book needs **2 pages of space (2KB)**.
- Shelf has **two empty spots of 1 page (1KB each)** at different places.

In old method (contiguous allocation):

Book must be placed **together**

- Not possible

With paging:

Tear the book into **two pages**

- Place **Page 1** in one empty spot
- Place **Page 2** in another empty spot

The book is **logically one**, but **physically spread out**.

What Paging Does

- Process is divided into **fixed-size pages** (e.g., 1KB).
- Memory is divided into **fixed-size frames** (also 1KB).
- Pages can be placed **anywhere** in free frames.
- OS keeps a **page table** to remember where each page is stored.

Why Paging Solves the Problem

- ✓ No need for contiguous memory
- ✓ External fragmentation is eliminated
- ✓ No heavy compaction required
- ✓ Better memory utilization

One-Line

Paging allows a process to live in pieces, just like a book whose pages are kept on different shelves — but the reader still reads it as one book.

3. Paging

a. What is Paging?

Paging is a method where a process does not need continuous memory.

The process can be stored in different places in RAM.

b. Why Paging?

Paging:

- **Removes external fragmentation**
- **No need to move memory again and again (no compaction)**

c. Basic Idea of Paging

Think of RAM like a parking area 🚗

- Parking area is divided into same-size slots → Frames
- Each car (process) is broken into same-size parts → Pages

📌 Page size = Frame size

Now:

- Any page can park in any free slot

d. Page Size

- Page size is decided by the CPU
- Common size = 4 KB
- New CPUs support multiple page sizes for better performance

e. Page Table

i. What is Page Table?

Page table is like a list that tells:

Page number → Frame number

ii. What does it store?

It stores the address of the frame where each page is kept.

📖 Like:

- Book = process
- Index page = page table
- Page number → shelf number

f. Logical Address

Every CPU address has two parts:

Page number + Offset

- Page number → find frame from page table
- Offset → exact position inside the frame

g. Where is Page Table Stored?

- Stored in main memory
- Created when process starts
- Address stored in PCB

h. PTBR (Page Table Base Register)

- PTBR points to current page table
- During context switch, OS only changes PTBR
  Fast switching

4. How Paging Removes External Fragmentation?

- Pages can be placed anywhere in memory
- Free memory can be scattered
- Still process fits properly

So no external fragmentation

5. Why Paging is Slow & How It Becomes Fast?

a. Why Paging is Slow?

Because:

1. First we read page table
2. Then we read actual data

Two memory accesses → slow

b. How Paging is Made Fast?

TLB (Fast Memory)

- TLB is a small and very fast memory
- Stores recent page-frame info

If found in TLB:

- ✓ Direct access
- ✓ Only one memory access

One Line

Paging allows memory to be used in pieces, removes external fragmentation, but uses TLB to stay fast.

6. Translation Look-aside Buffer (TLB)

a. What is TLB?

TLB is a hardware support used to make paging fast.

Paging slow hoti hai, isliye TLB use krte hai.

b. What kind of memory is TLB?

- TLB is a hardware cache
- It is very fast memory
- Much faster than main memory (RAM)

c. What does TLB store?

TLB stores key–value pairs:

- Key → Page Number
- Value → Frame Number

Direct mapping for quick access.

d. Why do we need TLB?

- Page table is stored in main memory
- Main memory access is slow

So when CPU generates an address:

1. First it checks page table (slow)
2. Then it accesses actual data (slow)

This makes address translation slow.

e. How TLB makes paging fast?

Process:

1. CPU generates a logical address
2. OS finds frame number using page table
3. This page → frame entry is stored in TLB
4. Next time:
    - CPU checks TLB first
    - If entry found → no page table access

Direct frame number

Faster address translation

f. TLB Hit

- If required page number is found in TLB
- This is called TLB hit

Result:

- No page table access
- Only one memory access

- Very fast

g. ASID (Address Space Identifier)

- Each TLB entry also stores an ASID
- ASID uniquely identifies a process

Why ASID is needed?

- TLB can store entries of multiple processes
- During lookup:
  - Page number must match
  - ASID must also match current process

If ASID does NOT match:

- Treated as TLB miss
- Page table is checked again

✓ Provides address space protection
✓ Allows multiple processes to share TLB safely

One-Line

TLB is a fast hardware cache that stores page-to-frame mappings to reduce page table access and speed up paging.